



دانشکده مهندسی کامپیوتر

باسمه تعالی

فرم پیشنهاد پروژه کارشناسی



دانشگاه صنعتی امیرکبیر
(پلی تکنیک تهران)

تحويل پیشنهاد پروژه به دانشکده و ثبت نهایی آن در پورتال: (این قسمت توسط کارشناسان آموزش دانشکده تکمیل می شود).

تاریخ تحويل پیشنهاد پروژه به آموزش دانشکده:

تاریخ ثبت نهایی در پورتال آموزشی دانشگاه:

مشخصات دانشجو:

نام و نام خانوادگی: علی ناصری نیا

شماره دانشجویی: ۴۰۱۳۱۰۵۳

رایانامه (ایمیل) دانشجو: alinaserinia@aut.ac.ir

نیمسال و سال تحصیلی ثبت نام پروژه: نیمسال اول از سال تحصیلی ۱۴۰۴

توضیح ۱: دانشجو موظف است حداکثر دو ماه پس از ثبت نام پروژه فرم تکمیل شده پیشنهاد پروژه را، که به امضای استاد راهنمای او رسیده است، به آموزش دانشکده تحويل دهد. انجام سر وقت این مرحله نشان دهنده بخشی از رعایت زمانبندی انجام پروژه توسط دانشجو است.

توضیح ۲: آموزش دانشکده پیشنهاد پروژه دریافتی را جهت تعیین داور و انجام داوری در اختیار گروه آموزشی استاد راهنمای دانشجو قرار میدهد. گروههای آموزشی حداکثر طی دو ماه داوری را انجام داده و در صورت تصویب در گروه، پیشنهاد پروژه را جهت تصویب در دانشکده و ثبت در پورتال آموزشی دانشگاه در اختیار آموزش دانشکده قرار میدهند. دانشجویان موظفند با داور(ان) پیشنهاد پروژه خود در ارتباط بوده و نظرات آنان را، با راهنمایی استاد راهنمای خود و در مهلت مقرر گروه برای تصویب پیشنهاد پروژه، بر روی پیشنهاد پروژه خود اعمال نمایند.

توضیح ۳: مهلت درج نموده پروژه دانشجویانی که در نیمسال اول یا در تابستان سال تحصیلی پروژه را اخذ نمودهاند، سی ام مهر سال تحصیلی بعد و برای دانشجویانی که در نیمسال دوم پروژه را اخذ نمودهاند، سی و یکام فروردین سال تحصیلی بعد است.

توضیح ۴: فاصله زمانی بین ثبت نهایی پیشنهاد پروژه (تصویب شده) در پورتال آموزشی دانشگاه و دفاع از پروژه حداقل سه ماه است و امکان دفاع قبل از سپری شدن این فاصله زمانی وجود ندارد. همچنین، دفاع از پروژه کارشناسی با اعلان عمومی و با حضور مخاطبان در حضور داوران انجام خواهد شد. لازم است دانشجویان حداقل سه هفته قبل از فرارسیدن مهلت درج نموده پروژه (توضیح ۳)، پایاننامه تایپ شده خود را، که به تأیید استاد راهنما رسیده است، در اختیار آموزش دانشکده و داور(ان) پروژه قرار داده و مقدمات برگزاری جلسه دفاع را، با هماهنگی آموزش دانشکده، فراهم آورند.

توضیح ۵: لازم است دانشجویان رویه دانشگاه صنعتی امیرکبیر با عنوان «چگونگی ثبت نام، تصویب، و دفاع از پایان نامه در مقطع کارشناسی» را که با شماره AUT-PR-3210 بر روی سایت معاونت آموزشی دانشگاه قرار گرفته است مطالعه کنند.

تاریخ:

امضای دانشجو:

استاد راهنمای پروژه:

تاریخ:

امضا:

نام و نام خانوادگی: حمیدرضا شهریاری کاهنکی

عنوان پروژه:

عنوان فارسی:

ترکیب تحلیل ساختاری کد با استدلال مدل‌های زبانی بزرگ و معماری چندعامله برای کاهش هشدارهای کاذب در تحلیل آسیب‌پذیری کد

عنوان انگلیسی:

Combining Code Structural Analysis with Large Language Model Reasoning and Multi-Agent Architecture to Reduce False Positives in Code Vulnerability Assessment

داور(ان) پیشنهاد پروژه:

داور اول:

نام و نام خانوادگی:

امضا:

تاریخ:

داور دوم:

نام و نام خانوادگی:

امضا:

تاریخ:

توضیح: با امضای این قسمت داور(ان) محترم تأیید میکنند که

- ۱- دانشجو، با راهنمایی استاد راهنمای خود، اصلاحات مورد نظر داور(ان) را انجام داده و عنوان و محتوای پیشنهاد پروژه از نظر ایشان قابل قبول است.
- ۲- دانشجو با مفاهیم پیشنیاز و مهارت‌های ضروری و پایه انجام این پروژه آشنایی داشته یا کسب آن برای دانشجو در طول انجام پروژه امکانپذیر است.
- ۳- موارد زیر در پیشنهاد پروژه مورد توجه قرار گرفته است:
 - عنوان پروژه به طور کامل و دقیق موضوع پروژه را نشان میدهد و محتوای پروژه با عنوان پروژه کاملاً مطابقت دارد.
 - پیشنهاد پروژه شامل بخشهای مقدمه، مرور پیشینه پژوهش، رویکرد پیشنهادی، روش ارزیابی، مراحل و زمانبندی انجام پروژه، امکانات لازم و لیست مراجع و منابع است.
 - اجزای سامانه مورد نظر پروژه در یک نمودار بلوکی نشان داده شده و ورودیها و خروجیهای آن مشخص شدهاند.
 - تأکید پروژه بر روی مسائل عملی و علمی و مهارت‌های مهندسی کامپیوتر است و پروژه منجر به توسعه نرم‌افزار، سخت‌افزار یا ترکیبی از آن دو و با درجه سختی و حجم مناسب یک پروژه سه واحدی است.
 - پروژه بر مبنای استفاده از دروس کارشناسی تعریف شده است.
 - چنانچه قرار است در پروژه از ابزارها، نرم‌افزارها، یا محیط‌های آماده استفاده شود، این موارد با صراحت بیان شده و مشخص شده است چه بخش‌هایی و با چه مقداری تلاش سهم دانشجو است.
 - پروژه علاوه بر بخش مطالعاتی-نظری، حدود ۱۵۰ ساعت کار عملی لازم داشته و انجام آن حداقل ۳ ماه زمان نیاز دارد.

تصویب پیشنهاد پروژه:

تصویب در گروه آموزشی:

نام و نام خانوادگی مدیر گروه:

امضا:

تاریخ:

تصویب در شورای آموزشی-پژوهشی دانشکده:

نام و نام خانوادگی معاون آموزشی:

امضا:

تاریخ:

تعریف پروژه: (دانشجو میتواند با اضافه کردن فاصله لازم بر روی فایل قابل ویرایش این سند، توضیحات خود را در هر یک از قسمتهای زیر تایپ کند.)

۱- مقدمه (بیان مسئله کاربردی، ضرورت، انگیزه، اهداف، و چالش‌های انجام این پروژه):

با رشد پیچیدگی و حجم کدهای نرم‌افزاری، روش‌های سنتی تحلیل ایستای کد^۱ با چالش‌های جدی مواجه شده‌اند. این ابزارها عمدتاً مبتنی بر قوانین نحوی و الگوهای از پیش تعریف‌شده هستند و در نتیجه، در بسیاری از موارد قادر به درک صحیح زمینه^۲ و منطق برنامه نیستند [۱]. این محدودیت باعث تولید نرخ بالایی از هشدارهای کاذب^۳ شده است و کارایی عملی آن‌ها را در پروژه‌های واقعی کاهش می‌دهد [۲].

در سال‌های اخیر، استفاده از مدل‌های زبانی بزرگ به‌عنوان ابزاری برای تحلیل معنایی کد مورد توجه پژوهشگران قرار گرفته است. این مدل‌ها توانایی استدلال محدود روی ساختار کد و توصیف رفتار آن را دارند و می‌توانند بخشی از خلأ موجود در ابزارهای تحلیل ایستای سنتی را پوشش دهند [۳]. با این حال، استفاده مستقیم و بدون چارچوب از این مدل‌ها ممکن است منجر به نتایج غیرقابل اعتماد و گزارش آسیب‌پذیری‌های غیرواقعی شود [۴].

هدف این پروژه، طراحی و پیاده‌سازی یک سامانه تحلیل ایستای هوشمند مبتنی بر عامل‌های هوشمند^۴ است که با بهره‌گیری از مدل‌های زبانی بزرگ^۵، توانایی استدلال معنایی محدود روی کد منبع^۶ را داشته باشد. این سامانه تلاش می‌کند نقش یک کارشناس امنیت نرم‌افزار را شبیه‌سازی کرده و با تحلیل ساختار نحوی^۷ و روابط ساده جریان‌داده در سطح درون‌تابعی^۸، تنها آسیب‌پذیری‌های معتبر را گزارش دهد.

تمرکز اصلی پروژه بر فاز تشخیص^۹ است و سامانه هیچ‌گونه تغییری در کد منبع اعمال نمی‌کند. خروجی نهایی، گزارشی تحلیلی از آسیب‌پذیری‌های شناخته‌شده (بر اساس CWE و CVE های منتخب) خواهد بود.

به‌منظور افزایش امکان‌پذیری و کاهش ریسک فنی، دامنه پروژه به‌صورت آگاهانه محدود می‌شود:

- تحلیل صرفاً به صورت ایستا و بدون اجرای کد انجام می‌شود.
- تحلیل معنایی محدود به سطح درون‌تابعی و رهگیری ساده جریان داده از منبع به نقطه مصرف^{۱۰}

زبان‌های برنامه‌نویسی هدف:

- C
- Python

تمرکز بر دسته‌ای مشخص از آسیب‌پذیری‌ها شامل:

- تزریق^{۱۱} شامل تزریق فرمان^{۱۲}، تزریق قالب^{۱۳} و تزریق کد^{۱۴}
- سرریز بافر^{۱۵} و سرریز عددی^{۱۶} در زبان C
- غیربازسازی^{۱۷} نا امن و استفاده خطرناک از رابط‌های برنامه‌نویسی کاربردی^{۱۸} در زبان Python [۵]

¹ Static Application Security Testing (SAST)

² Context

³ False Positives

⁴ Intelligent Agents

⁵ Large Language Models (LLMs)

⁶ Source Code

⁷ Abstract Syntax Tree (AST)

⁸ Intra-procedural

⁹ Detection

¹⁰ Source → Sink

¹¹ Injection

¹² Command Injection

¹³ Template Injection

¹⁴ Code Injection

¹⁵ Buffer Overflow

¹⁶ Integer Overflow

¹⁷ Deserialization

¹⁸ Application Programming Interface (API)

۲- مروری بر پروژه‌ها و سامانه‌های مشابه و بیان نقاط قوتی که با انجام این پروژه حاصل میشود:

ابزارهای رایج مانند Bandit (برای Python) و Flawfinder (برای C) مبتنی بر قوانین ایستا هستند. اگرچه این ابزارها سریع‌اند، اما فاقد درک معنایی از منطق برنامه بوده و در سناریوهایی مانند توابع سفارشی پاک‌سازی ورودی یا مسیرهای شرطی پیچیده دچار خطا می‌شوند. از سوی دیگر، استفاده مستقیم از LLM ها برای بررسی کد بدون ساختار تحلیلی مشخص، منجر به تولید نتایج غیرقابل اعتماد و پدیده «توهم مدل» می‌شود. خلاصه‌ای پژوهشی که این پروژه هدف قرار می‌دهد، ترکیب تحلیل ساختاری کد با استدلال کنترل‌شده LLM و استفاده از معماری چندعامله برای کاهش هشدارهای کاذب است. نوآوری‌ها و نقاط قوت پروژه

- معماری دو-عامله کنترل‌شده: سامانه شامل دو عامل مستقل است:
 - عامل جستجوگر^{۱۹}: شناسایی اولیه الگوهای بالقوه آسیب‌پذیر
 - عامل ارزیاب^{۲۰}: رد یا تأیید یافته‌ها صرفاً بر اساس شواهد صریح جریان داده
- استدلال مبتنی بر شواهد: هر آسیب‌پذیری تنها در صورتی گزارش می‌شود که مسیر مشخص جریان داده از منبع به نقطه مصرف ارائه گردد.
- کاهش هشدار کاذب: عامل ارزیاب به‌صورت پیش‌فرض رویکرد «رد مگر اثبات شود» را دارد.
- گزارش‌دهی تحلیلی: خروجی شامل توضیح متنی کوتاه و قابل فهم از منطق آسیب‌پذیری است.

۳- روش انجام پروژه (روش، نمودار بلوکی اجزای سامانه‌ی مورد نظر پروژه، ورودی‌ها و خروجی‌ها):

پیاده‌سازی سامانه با استفاده از زبان Python انجام می‌شود. مراحل اصلی به شرح زیر است:

پیش‌پردازش کد

- تجزیه کد منبع با استفاده از Tree-sitter
- استخراج توابع، پارامترها و فراخوانی‌های حساس
- تولید نمایش ساده AST برای استفاده عامل‌ها

عامل جستجوگر

- دریافت ساختار کد و توابع حساس
- شناسایی سناریوهای بالقوه آسیب‌پذیر بر اساس CVE های منتخب
- تولید فرضیه آسیب‌پذیری همراه با شواهد اولیه

عامل ارزیاب

- بررسی منطقی فرضیه‌ها
- رد موارد فاقد مسیر مشخص جریان داده
- تأیید نهایی آسیب‌پذیری‌های معتبر

خروجی سامانه

- فایل گزارش در قالب JSON
- شامل: نوع آسیب‌پذیری، CWE مرتبط، CVE مرجع، محل دقیق کد و توضیح تحلیلی
- آسیب‌پذیری‌ها و CVE های مورد بررسی:

• زبان C

- CVE-2021-3156: سرریز بافر در نرم‌افزار Sudo
- CVE-2017-7529: سرریز عددی در وب‌سرور Nginx

• زبان Python

- CVE-2021-25239: تزریق قالب در سمت سرور^{۲۱} در Flask/Jinja2
- CVE-2022-22817: غیربازسازی ناامن داده‌ها^{۲۲} در کتابخانه PyYAML

این CVE ها به دلیل وضوح الگوی کد و امکان تشخیص ایستا انتخاب شده‌اند.

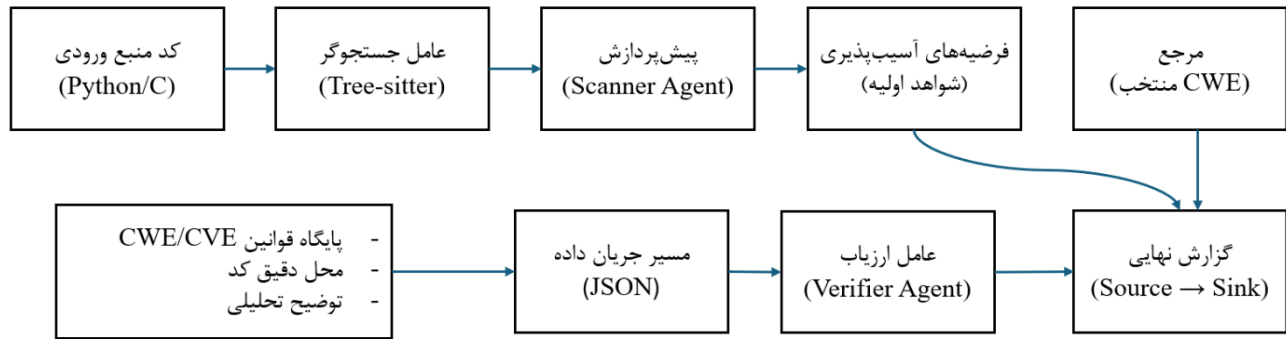
¹⁹ Scanner Agent

²⁰ Verifier Agent

²¹ Server-Side Template Injection

²² Unsafe Deserialization

در ادامه نمودار بلوکی روش انجام پروژه آمده است: (شکل ۱.۳)



شکل ۱.۳

۴- روش ارزیابی:

ارزیابی سامانه به روش های زیر انجام می شود:

۱. استفاده از دیتاست های استاندارد

- SARD (NIST): شامل بیش از ۱۰۰۰۰۰ نمونه کد آسیب پذیر و سالم در زبان های C و Python. برای هر نمونه نوع دقیق CWE مشخص شده است.
- Juliet Test Suite: پوشش ۱۱۸ دسته CWE از جمله تزریق، سرریز بافر، غیربازسازی ناامن، مناسب برای سنجش Precision و Recall.
- Devign: شامل ۲۸۰۰۰ تابع واقعی از لینوکس کرنل با برچسب آسیب پذیری.
- Big-Vul: شامل ۳۷۵۵ آسیب پذیری واقعی با CVE مرتبط و commit اصلاحی

۲. مقایسه نتایج با ابزارهای سنتی:

- Bandit برای Python
- Flawfinder برای C

۳. محاسبه معیارهای Precision و Recall

۴. بررسی دستی نمونه های از گزارش ها برای ارزیابی کیفیت توضیحات

۵- مراحل انجام و زمانبندی پروژه:

مراحل پیاده سازی به صورت زیر است (جدول ۱.۵):

زمان بندی	توضیحات	مراحل
۳ تا ۴ هفته	بررسی و یادگیری مفاهیم مورد نیاز	تحقیق و مطالعه
۳ تا ۴ هفته	بررسی مدل ها و روش های پیاده سازی و انتخاب یکی از این روش ها	طراحی مدل و انتخاب روش پیاده سازی
۲ تا ۳ ماه	پیاده سازی و کدنویسی بخش های مختلف و طراحی پرامپت ها	پیاده سازی ابزار
۱ تا ۲ ماه	ارزیابی ابزار و رفع اشکالات احتمالی	ارزیابی و بهبود
۱ تا ۲ هفته	نگارش گزارش نهایی	مستند سازی

جدول ۱.۵: جدول زمان بندی پروژه

۶- امکانات لازم (ابزارها، محیط‌ها، و نرم‌افزارهای مورد استفاده):

ابزار و محیط مورد نیاز برای پیاده‌سازی سامانه بدین صورت است:

- زبان برنامه‌نویسی: Python
- ویرایشگر^{۲۳} مناسب کد مانند Visual Studio Code
- کتابخانه‌ها: Tree-sitter، LangChain یا AutoGen
- مدل زبانی: API مدل‌های LLM یا مدل متن‌باز
- سخت‌افزار: سیستم معمولی (در صورت استفاده از API)
- محیط لینوکس^{۲۴} برای توسعه ابزار

۷- مراجع و منابع:

- [۱] B. Johnson, Y. Song, E. Murphy-Hill, and R. Bowdidge, Why Don't Software Developers Use Static Analysis Tools to Find Bugs?, Proceedings of the 35th International Conference on Software Engineering (ICSE), 2013 .
- [۲] N. Ayewah and W. Pugh, The Google FindBugs Fixit, Proceedings of the 19th International Symposium on Software Testing and Analysis (ISSTA),, 2010 .
- [۳] Z. Chen, S. Fan, Y. Zhao, and X. Liu, Large Language Models for Code Analysis: A Survey, arXiv preprint arXiv:2308.12345, 2023 .
- [۴] S. Zhang, H. Chen and Y. Zhou, On the Reliability of Large Language Models for Software Vulnerability Detection, arXiv preprint arXiv:2309.04512, 2023 .
- [۵] Y. Zhou, S. Liu, J. Siow, X. Du and Y. Liu, “Devign: Effective Vulnerability Identification by Learning Comprehensive Program Semantics via Graph Neural Networks ”, *Advances in Neural Information Processing Systems (NeurIPS)* .۲۰۱۹ ,
- [۶] D. Evans and D. Larochelle, “Improving Security Using Extensible Lightweight Static Analysis ”, *IEEE Software*, vol. 19, no. 1 ,p. 42–51, 2002 .
- [۷] F. Wu, X. Wang, Y. Liu, and Z. Zhang, AutoAudit: Automated Code Auditing with Multi-Agent Systems, arXiv preprint arXiv:2403.01234, 2024 .
- [۸] M. Corporation, Common Weakness Enumeration (CWE), <https://cwe.mitre.org/>, 2025 .

۸- پیوست‌ها:

<https://tree-sitter.github.io/tree-sitter/>
<https://microsoft.github.io/autogen/>
<https://www.langchain.com/docs/>
<https://platform.openai.com/docs/>
<https://github.com/fuxudong/Devign>
<https://samate.nist.gov/SARD/>
<https://cwe.mitre.org/>
<https://cve.mitre.org/>

²³ IDE

²⁴ Linux